Ex.No. :04

Create UDP and TCP based Network Applications using Socket Programming

Aim

To develop Client Server Applications using TCP and UDP sockets

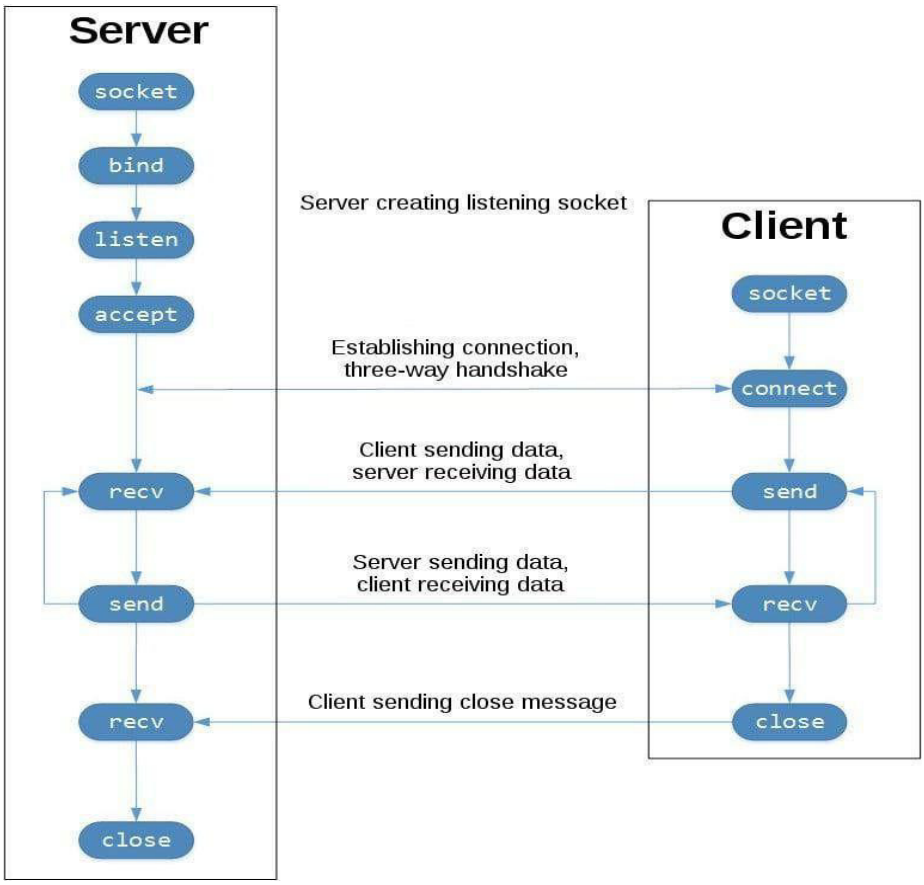
Echo Client-Server program using TCP sockets :

Steps in Server:

1. Call socket() method with SOCK_STREAM option to create TCP socket
2. Call bind() method which binds it to a specific IP and port so that it can listen to incoming requests on that IP and port.
3. Call listen() method which puts the server into listening mode. This allows the server to listen to incoming connections.
4. Call accept() method to initiate a connection with the client
5. Finally call close() method closes the connection with the client.

Steps in Client :

1. Call socket() method with SOCK_STREAM option to create TCP socket
2. Call connect() method to initiate a connection with the server
3. Finally call close() method closes the connection with the client.


Server
socket
bind
listen
accept
recv
send
recv
close
Server creating listening socket
Establishing connection,
three-way handshake
Client sending data,
server receiving data
Server sending data,
client receiving data
Client sending close message
Client
socket
connect
send
recv
close

```
# echo-server.py

import socket

HOST = "127.0.0.1"  # Standard loopback interface address (localhost)
PORT = 65432  # Port to listen on (non-privileged ports are > 1023)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    conn, addr = s.accept()
    with conn:
        print(f"Connected by {addr}")
        while True:
            data = conn.recv(1024)
            if not data:
                break
            conn.sendall(data)

# echo-client.py
import socket

HOST = "127.0.0.1"  # The server's hostname or IP address
PORT = 65432  # The port used by the server

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b"Hello, world")
    data = s.recv(1024)

print(f"Received {data!r}")
```

Sample Input and Output

`$ python_echo-server.py` `Connected by ('127.0.0.1', 64623)`	`$ python_echo-client.py` `Received b'Hello, world`

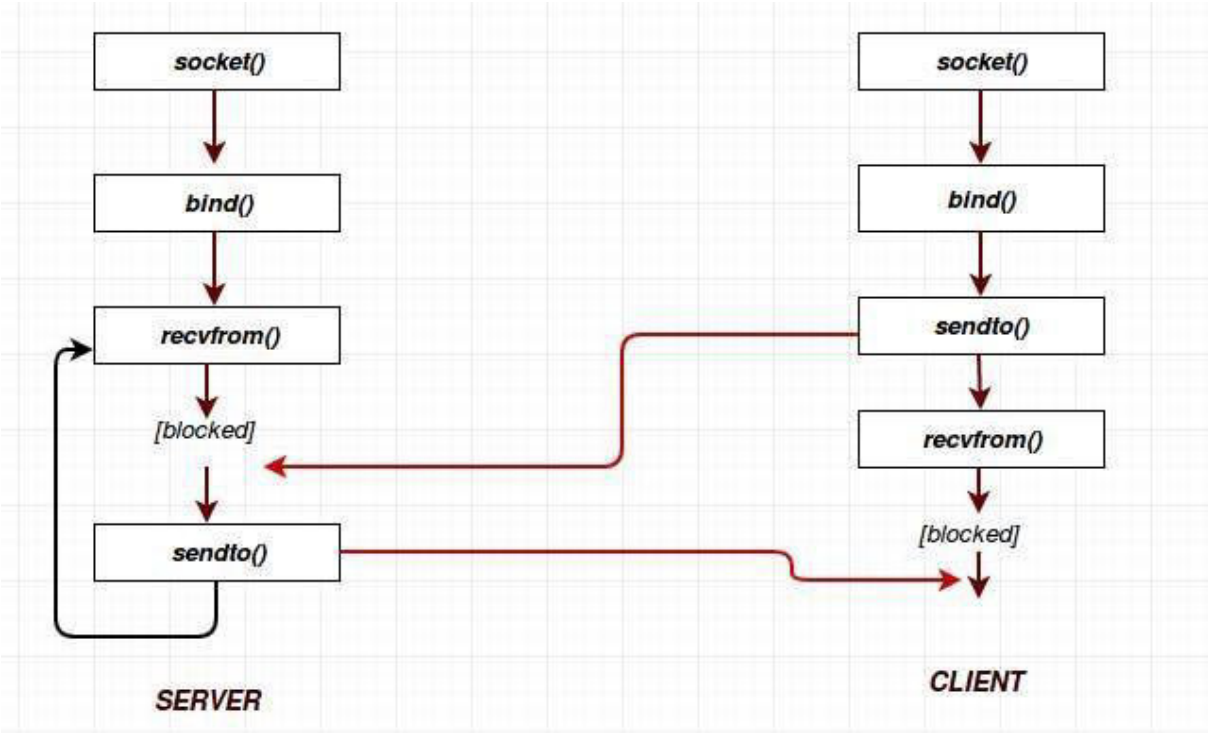
Echo Client-Server program using UDP sockets :

Steps in Server:

1. Call socket() method with SOCK_DGRAM option to create UDP socket
2. Call bind() method which binds it to a specific IP and port so that it can listen to incoming requests on that IP and port.
3. Call recvfrom() method to receive the data from the client.
4. Call sendto() method to send the reply to the client

Steps in Client **:**

1. Call socket() method with SOCK_DGRAM option to create UDP socket
2. Call sendto() method to send request to the server
3. Call recvfrom method to get the reply from the server.



#udpserver.py

```
import socket
sock = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)
udp_host = socket.gethostname()                    # Host IP
udp_port = 12345                                   # specified port to connect
sock.bind((udp_host,udp_port))
while True:
        print "Waiting for client...."
        data,addr = sock.recvfrom(1024)            #receive data from client
```

```
        print "Received Messages:",data," from",addr
        sock.sendto(data,(addr))
```

#udpclient.py

```
import socket
sock = socket.socket(socket.AF_INET,socket.SOCK_DGRAM)      # For UDP
udp_host = socket.gethostname()         # Host IP
udp_port = 12345                          # specified port to connect
msg = "Hello Python!"
print "UDP target IP:", udp_host
print "UDP target Port:", udp_port
sock.sendto(msg,(udp_host,udp_port))
data,addr = sock.recvfrom(1024)           #receive data from server
print "Received Messages:",data," from",addr
```

Sample Input and Output

`$ python_echo-server.py`
`Waiting for client`
`Received messages: Hello Python!`
	`$ python_echo-client.py`
	UDP target IP : 127.0.0.1
	UDP target Port : 12345
	`Received messages: Hello Python!`

Result:

Thus, capturing and analyzing DNS packets using Wireshark has been completed successfully.